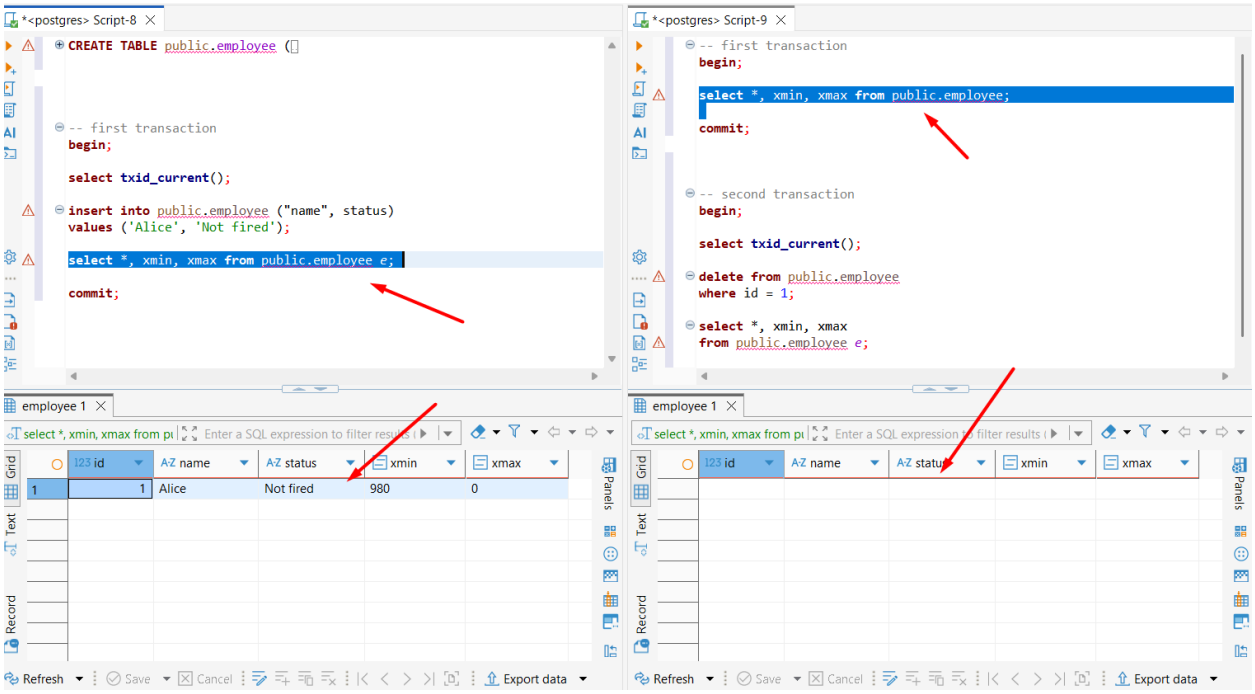
!NOTE:

The 1st transaction is called the one shown on the left in the screenshots, and the 2nd is called the one on the right.

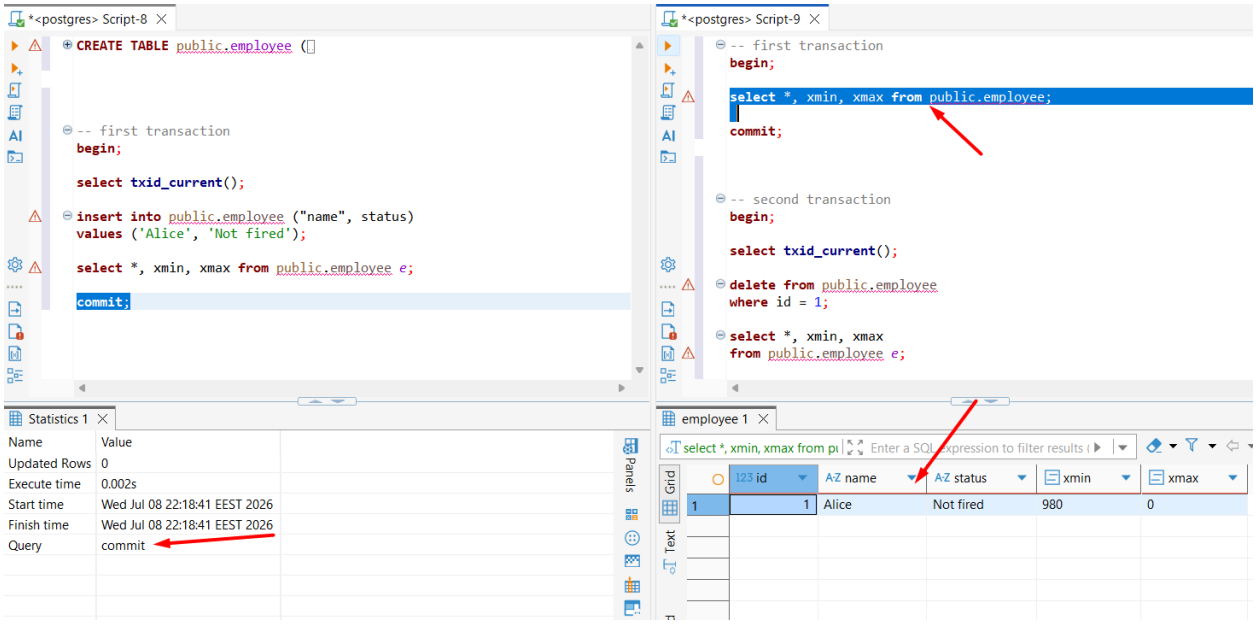
–Create a table called employee with columns id serial, name varchar, status varchar.
–Replicate the example given in the lecture with the code:

– 1st scenario

Two separate transactions are started. The insert occurred in the 1st transaction that is not visible in the 2nd transaction

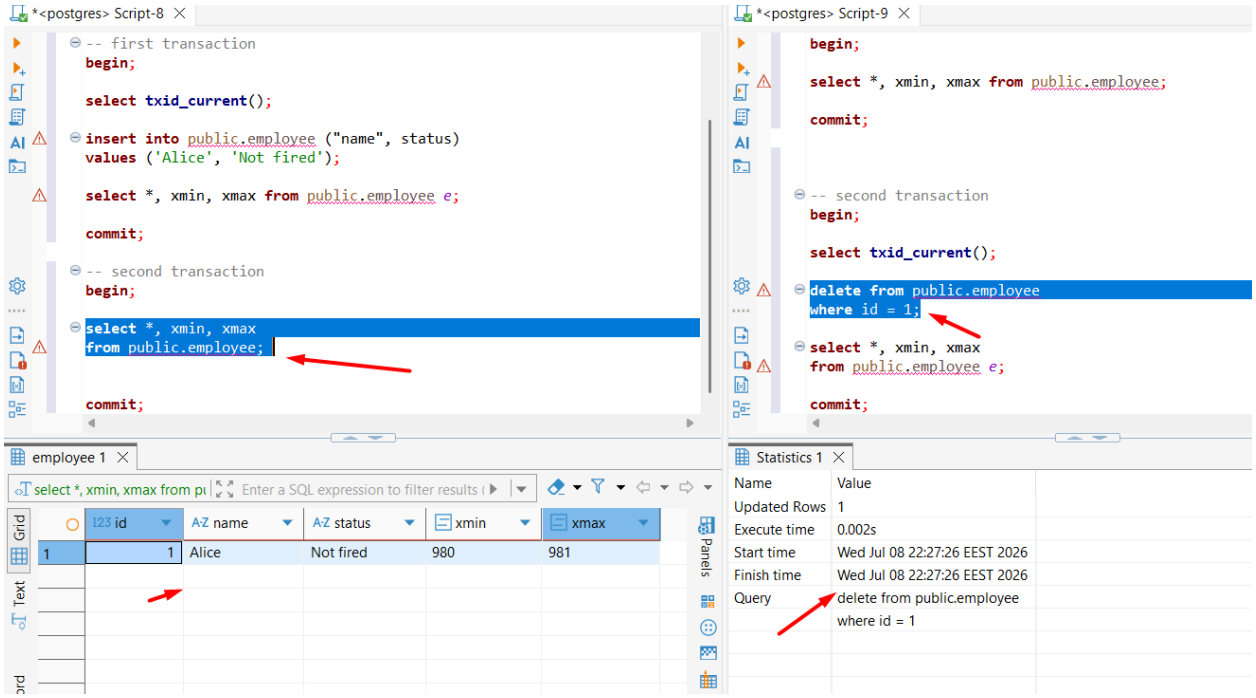


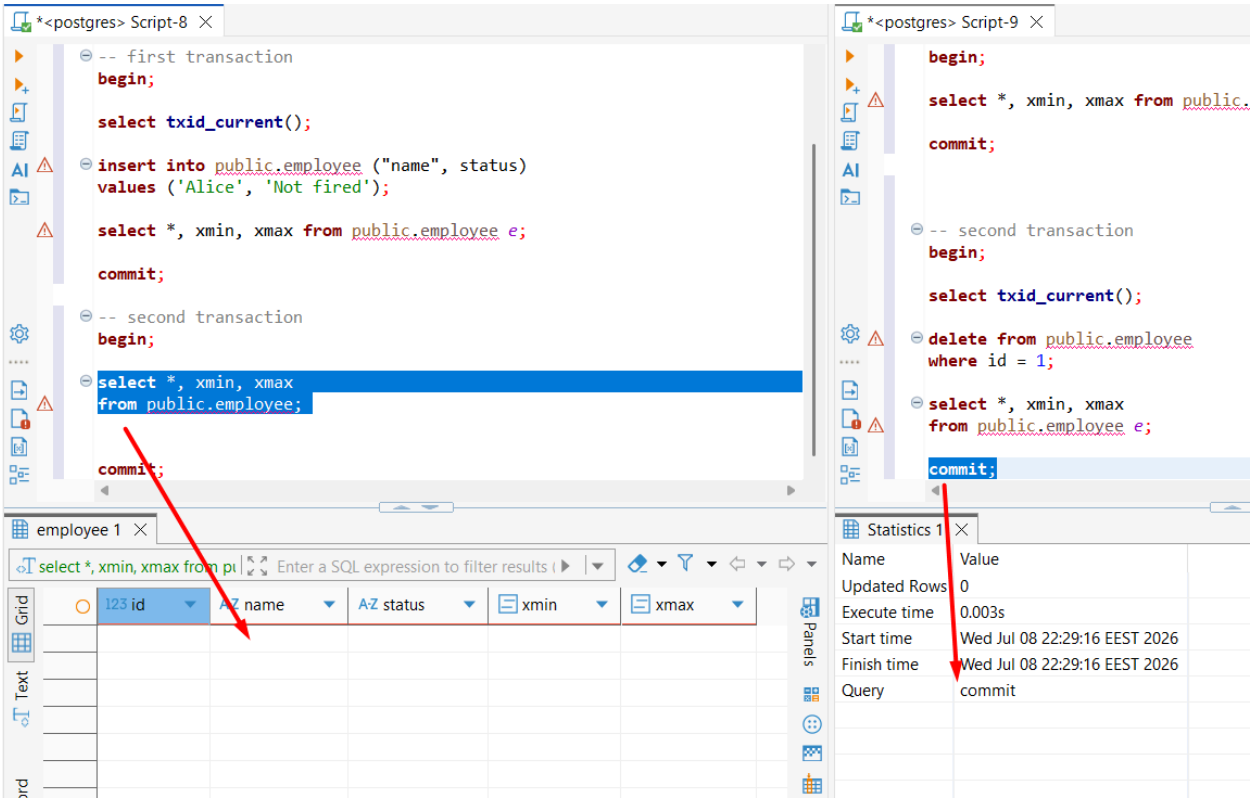
After the committing 1st transaction, the inserted row is appeared in the second transaction - **non-repeatable read anomaly** happened in the scope of 2nd transaction - so, we used **'Read committed'** isolation level



– 2nd scenario

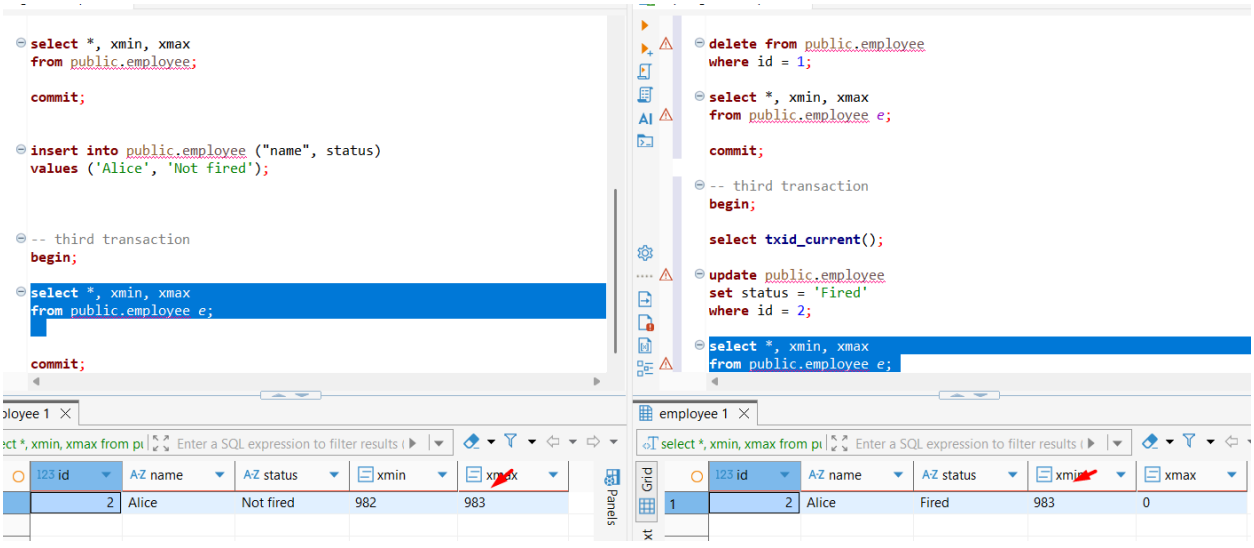
The row was deleted in the scope of uncommitted 2nd transaction, so it is still visible on the SELECT command of 1st transaction and disappeared after the 2nd transaction commit



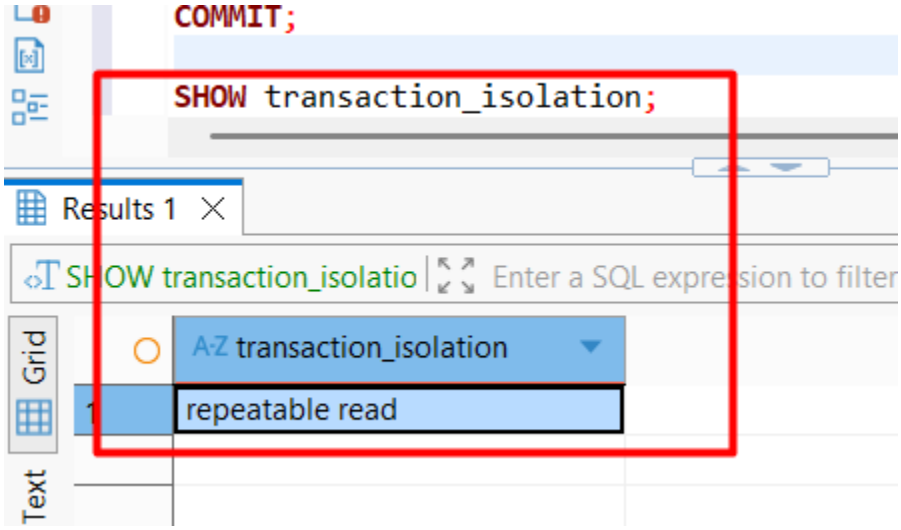


– 3rd scenario

The 2nd transaction updates the table data change is not reflected in the scope of 1st transaction before commit but displayed after the commit



–Run the command set transaction isolation level repeatable read.

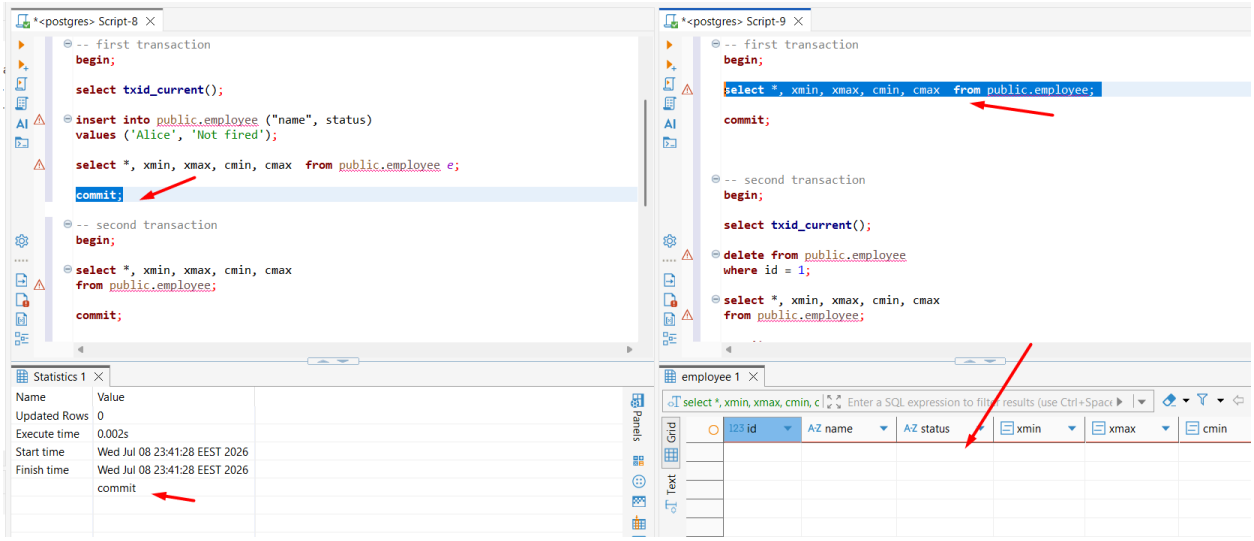


–Check your current isolation level in each session with show transaction isolation level.

– 1st scenario

Two separate transactions are started. The insert occurred in the 1st transaction that is not visible in the 2nd transaction

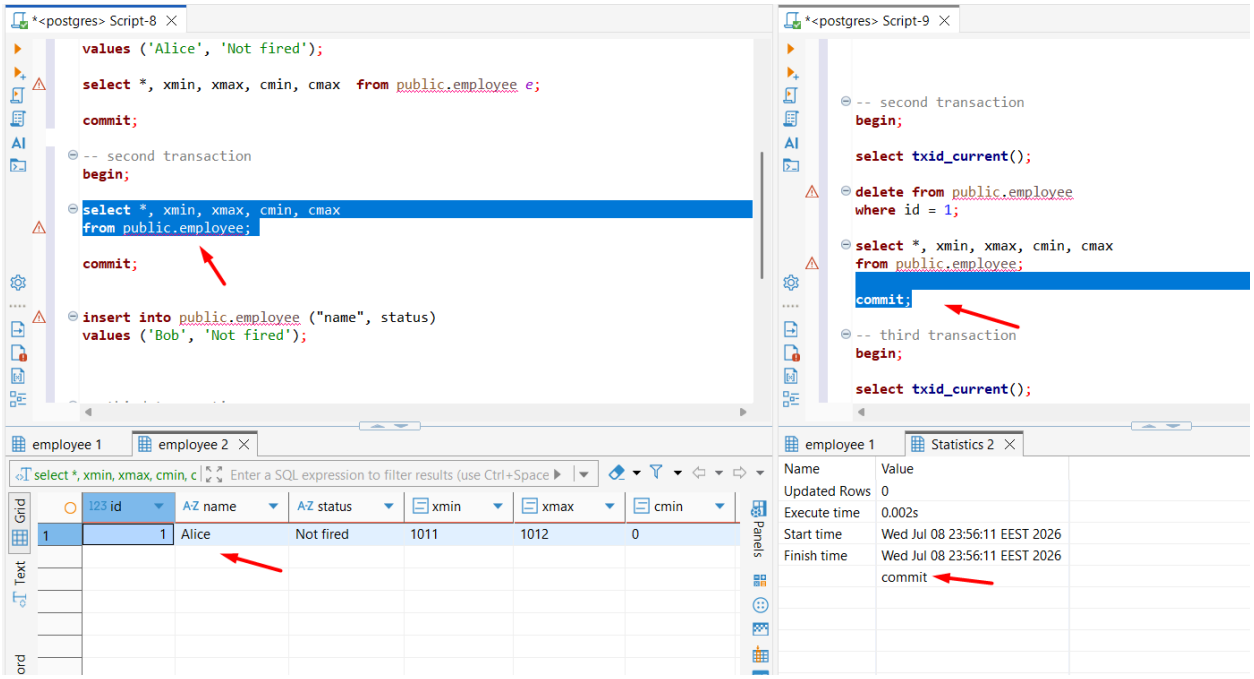
After the committing 1st transaction, the inserted row is **NOT** appeared in the second transaction - **non-repeatable read anomaly** is **gone** in the scope of 2nd transaction - so, we used **'Repeatable read'** isolation level



– 2nd scenario

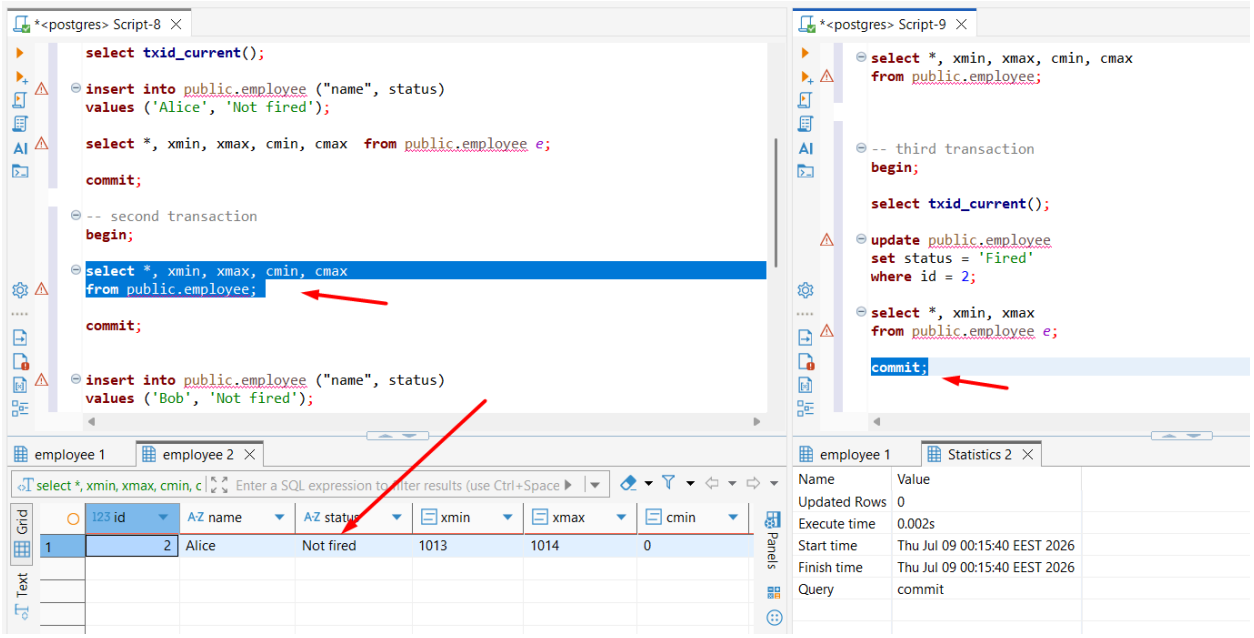
The row was deleted in the scope of uncommitted 2nd transaction, so it is still visible on the SELECT command of 1st transaction and **NOT** disappeared after the 2nd transaction commit

(1st is still uncommitted)



– 3rd scenario

The 2nd transaction updates the table data change is not reflected in the scope of 1st transaction before and **after** commit (1st is still uncommitted)



–Recreate employee table and redo the second task but modify the code so that select statements would now include cmin and cmax system columns. What changed?

Nothing changed when the tasks flow repeated. Because we had one query per transaction. To see these columns change, a few inserts within 1 transaction were performed.